

SQL 漏洞报告

报告日期: 2026-07-08

漏洞类型: SQL 注入 (UNION 注入 + OR 万能注入)

测试工具: curl / Burp Suite

修复状态: 已修复

一、漏洞概述

Flask 用户管理系统的搜索功能使用 f-string 直接拼接用户输入到 SQL 语句中，未做任何过滤或转义，导致存在 SQL 注入漏洞。

漏洞代码

```
sql = f"SELECT * FROM users WHERE username LIKE '%{keyword}%' OR email LIKE '%{keyword}%'  
cursor.execute(sql)
```

攻击者可通过构造特殊输入闭合字符串，改变 SQL 语句的逻辑，从而获取全部用户数据。

二、UNION 联合注入

注入原理

UNION 注入通过闭合原 SQL 语句，使用 UNION SELECT 追加自定义查询结果。

关键步骤一：探测列数

输入: ' UNION SELECT 1,2,3,4,5--

生成 SQL:

```
SELECT * FROM users
WHERE username LIKE '%' UNION SELECT 1,2,3,4,5--%' OR email LIKE '%' UNION SELE
```

-- 注释掉后面所有内容，实际执行：

```
SELECT * FROM users WHERE username LIKE '%' UNION SELECT 1,2,3,4,5
```

结果：确认 `users` 表共有 5 列（id, username, password, email, phone）。

关键步骤二：插入虚假数据

```
输入： ' UNION SELECT 1,'inj','p','inj@x.com','138'--
```

结果：搜索结果中出现伪造用户 `inj`，与真实用户数据混在一起显示。

关键步骤三：提取全部数据

```
输入： ' UNION SELECT id,username,password,email,phone FROM users--
```

结果：可提取数据库中全部用户的所有字段。

三、OR 万能注入

注入原理

OR 注入通过让 WHERE 条件恒为真，绕过搜索限制返回全表数据。

```
输入： ' OR '1'='1
```

生成 SQL：

```
SELECT * FROM users
WHERE username LIKE '%' OR '1'='1%' OR email LIKE '%' OR '1'='1%'
```

由于 `username LIKE '%'` 恒为真（% 匹配任意字符串），全部用户被返回。

实际测试结果：共泄露 7 个用户 (admin, alice, testuser, testuser2, newuser, zhangsan, hacker) 。

原理拆解

条件	含义	结果
username LIKE '%'	% 匹配任意字符串	恒真，返回全部用户
'1'='1%'	字符串比较	恒假
OR 连接	任一条件为真即返回	整体结果：返回全表

四、修复方案

使用参数化查询（已实施）

修复后在 process/app.py 中把所有 SQL 查询改为 ? 占位符形式：

```
# 修复前（存在注入）
sql = f"SELECT * FROM users WHERE username LIKE '%{keyword}%' OR email LIKE '%{'

# 修复后（安全）
cursor.execute("SELECT * FROM users WHERE username LIKE ? OR email LIKE ?",
               (f"%{keyword}%", f"%{keyword}%"))
```

参数化查询将 SQL 逻辑与用户数据分离，数据库驱动自动对用户输入中的特殊字符进行转义处理。

修复验证

测试 payload	修复前	修复后
' OR '1'='1	❌ 泄露全部用户	✅ 仅搜索字面字符串
' UNION SELECT 1,2,3,4,5--	❌ 注入成功	✅ 仅搜索字面字符串

测试 payload	修复前	修复后
正常搜索 keyword=admin	✅ 正常	✅ 正常

五、结论

本次发现的两个 SQL 注入漏洞（UNION 注入、OR 万能注入）均源于 f-string 拼接 SQL 语句，现已通过参数化查询完成修复。修复后攻击者无法通过搜索功能注入或提取任意数据。